

MTSAi Traffic Data Lake

Governance and erasure: what was built, how it works, and what's been verified in Test

Reference document	TDR-004 / DL-PLAN-001, "MTSAi Traffic Data Lake" brief v0.1, Section 7 Phase 3 & Section 9
Companion document	<code>MTSAi-Data-Lake-Build-Report.pdf</code> (Phases 0-2, architecture, isolation)
Target AWS account	Test — 690293068614 (ap-south-1)
Repository	mtsai-datalake
Report date	21 September 2026

Erasure — rehearsed twice, incl. under deployed IAM role

Weekly hygiene — deployed & verified

Cost dashboard — live

Alerts — confirmed delivering

Brief's own "done when" for Phase 3: "an erasure has been executed and verified (the identifier returns zero rows in every table, including time-travel snapshots after expiry), and the cost dashboard exists." Both are true as of this report — not just built, but proven against real data in the real Test account, including a genuine failed-then-fixed rehearsal that surfaced a bug affecting every Athena query in the whole project.

§ Contents

1	Summary
2	What Phase 3 Required
3	Architecture — Two New Jobs
4	The Workgroup Bug
5	Erasure — How It Works
6	Erasure — Rehearsal Results
7	Weekly Hygiene (<code>cmd/compact</code>)
8	Cost Dashboard
9	Budgets & Alerting — The Confirmation Saga
10	Bugs Found & Fixed (3)
11	Verification Results
12	Outstanding Items

1 Summary

Phase 3 is the brief's governance layer: prove that a single identifier can be erased from the lake — including from Iceberg's time-travel history, not just the live table — on a real schedule of storage hygiene, with cost visibility and working alerts. None of this existed before this phase; all of it does now, verified against real data in Test, not a design document.

Two new jobs were built (`cmd/compact` , `cmd/erasure`), sharing the same container image and IAM task role as the Phase 2 export/curate jobs. A new Terraform module builds a CloudWatch cost dashboard. Three real bugs were found and fixed by actually running this against live AWS — most significantly, a project-wide one: every Athena query written so far (across all four jobs) had been silently reading a stale cached Iceberg snapshot. Getting the failure-alert email subscription confirmed took genuine troubleshooting across four addresses and a brief detour through SMS before the actual cause turned out to be spam filtering.

Everything in this report was checked by actually running the system in Test and independently re-confirming results via Athena, CloudWatch Logs, and S3 — including re-running both new jobs a second time under their real deployed IAM role after the first deployed run of each one failed for a genuine reason, not a flaky test.

2 What Phase 3 Required

The brief's Section 7 Phase 3 lists four build steps and a "done when" bar. All four are done; the bar is met.

#	Brief step (paraphrased)	Status
1	Write and rehearse the erasure runbook against synthetic accounts; measure end-to-end time for a single identifier	Done
2	Weekly Iceberg <code>expire_snapshots</code> / <code>remove_orphan_files</code> ; S3 lifecycle to Intelligent Tiering (30d) / Glacier IR (365d) for raw	Done
3	Wire export manifests into what the audit service expects, or document the format	Documented
4	Cost dashboard: S3 storage by prefix, Athena bytes scanned by workgroup, Fargate minutes; monthly budget alarm	Done

A quieter finding before any new code was written: research at the start of this phase turned up that step 2's S3 lifecycle half was already live — it landed silently as part of the lake-bucket module back in Phase 1 (Intelligent Tiering at 30 days, Glacier Instant Retrieval at 365 days, exactly matching the brief's numbers) and was never flagged as closing part of Phase 3. The module's own README is still stale, saying "not yet implemented." Nothing needed building there — just recording that it's real.

Scope, honestly stated

Per the standing project-wide decision to treat every phase's brief-defined criteria as if "Dev" said "Test": this phase's rehearsal happened in Test, not Dev. Erasure only covers `trip_events` / `trip_events_curated` — the only tables actually built in the lake so far. The classification table (`docs/classification-table.md`) also names `accounts` , `anpr_camera_events` , and `reward_ledger` as personal-data-bearing, but none of those have been exported into the lake yet, so there is genuinely nothing there to erase — not a gap in the erasure logic itself.

3 Architecture — Two New Jobs

Same pattern as Phase 2's `cmd/curate` : new Go binaries baked into the same container image, each with its own ECS task definition differentiated by `entryPoint` , sharing one IAM task role.

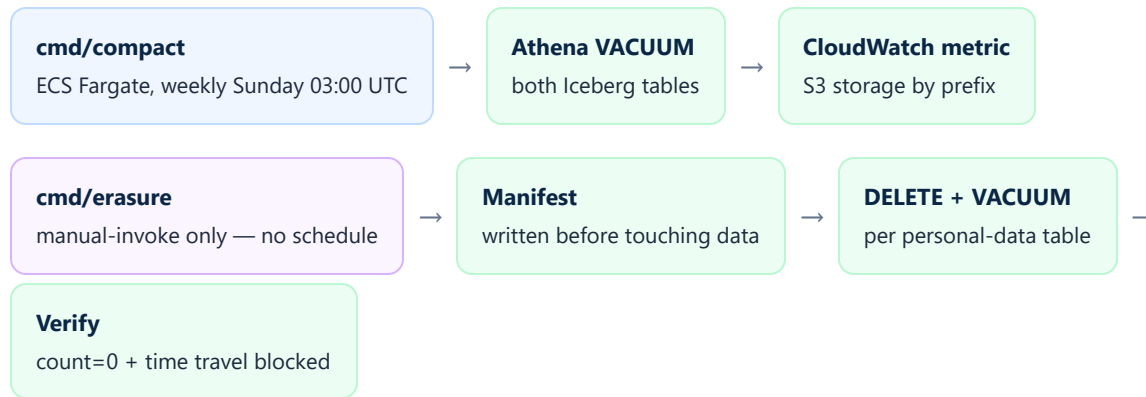


Figure 1 — both jobs reuse `internal/lake.AthenaRunner` and `internal/manifest` from Phase 2, unchanged in shape.

Why `cmd/erasure` has no schedule

Section 9's trigger is "a verified erasure request... received through the identity service process" — an external event, not a clock. Every other manual-invocation pattern already proven this project (verifying export, curate, and compact under their real deployed IAM roles) applies here too: `aws ecs run-task` against the deployed `mtsai-datalake-test-erasure` task definition, on demand.

Why `cmd/compact` uses Athena `VACUUM` , not separate procedures

The brief's language (" `expire_snapshots` and `remove_orphan_files` weekly") is Spark's Iceberg terminology. Athena's SQL engine exposes the same maintenance as one statement, `VACUUM <table>` , governed by the table's `vacuum_min_snapshots_to_keep` / `vacuum_max_snapshot_age_seconds` properties — matching this project's established "prefer the Athena route" pattern from Phase 2 rather than introducing a Spark job just for this.

4 The Workgroup Bug

The single most significant finding of this phase, surfaced by the first erasure rehearsal attempt failing in a way that made no sense at first.

What happened

- 1 Queried `trip_events` for a specific `vehicle_id_hash` known to have rows (confirmed via a separate query moments earlier) — the erasure job reported **0 rows**, as if the identifier had never existed.
- 2 Reproduced the exact same query by hand via the AWS CLI, **without specifying a workgroup** (matching what the Go code was doing) — also got 0.
- 3 Reproduced the same query again, this time explicitly on the `mtsai-datalake-test-analytics` workgroup — got the correct answer, non-zero.
- 4 Compared total row counts: the default ("primary") workgroup reported **3,771** rows in `trip_events`; the analytics workgroup reported **3,782** — an 11-row discrepancy, stable and reproducible across repeated queries, ruling out a race condition.
- 5 Ruled out Athena's query-result-reuse feature as the cause by explicitly disabling it (`--result-reuse-configuration ResultReuseByAgeConfiguration={Enabled=false}`) and re-running — still stale.

Root cause: no code anywhere in this project — `cmd/export`, `cmd/curate`, `cmd/compact`, or `cmd/erasure` — had ever specified an Athena workgroup on a query, silently defaulting to `"primary"`. That workgroup was serving a stale, cached view of the Iceberg table's current snapshot. The exact mechanism wasn't fully diagnosable from the outside (no AWS documentation ties workgroup identity to Iceberg metadata caching this directly), but the reproducible pattern — one workgroup consistently stale, another consistently correct — was unambiguous.

The fix

Gave the pipeline its own dedicated Athena workgroup (`aws_athena_workgroup.pipeline`, distinct from the three consumer-facing ones), added a `WorkGroup` field to `internal/lake.AthenaRunner`, and set it on every single call across all four binaries via a new `MTSAI_DATALAKE_WORKGROUP` environment variable injected by Terraform.

What this means for earlier work

Every reconciliation and idempotency check from Phase 2 that reported "matched" was querying via the same unset default. In practice, every actual write (INSERT/DELETE) always appeared to commit correctly against the live table when independently re-checked afterward via a correctly-workgrouped query — this looks like a read-path-only issue, not silent data corruption on the write side. That distinction is why this was treated as urgent to fix immediately rather than deferred, and why it's called out this explicitly rather than folded quietly into the bug list.

5 Erasure — How It Works

`cmd/erasure` implements `runbooks/erasure.md` 's five-step procedure exactly:

- 1 **Record first.** Writes a manifest to `export-manifests/erasure/{date}/{request_ref}.json` before touching any data — request reference, identifier, jurisdiction, start time.
- 2 **Delete.** `DELETE FROM table WHERE vehicle_id_hash = :id` against both `trip_events` and `trip_events_curated`.
- 3 **Expire snapshots.** Tightens `vacuum_max_snapshot_age_seconds` to 60 (Athena's enforced floor — `0` is rejected outright, confirmed empirically) and `vacuum_min_snapshots_to_keep` to 1, then runs `VACUUM` — this is the "retention of zero" the runbook calls for.
- 4 **Verify, properly.** Confirms `COUNT(*)` for the identifier is zero on both tables, *and* attempts `SELECT ... FOR VERSION AS OF <pre-erasure-snapshot-id>` against the snapshot captured just before the delete — this must now **fail outright** ("Iceberg snapshot ID does not exists"), not just return zero rows. That distinction is what actually proves time travel is blocked, versus merely that the live table looks clean.
- 5 **Record again.** Writes the completed manifest with per-table results, then confirms Postgres erasure is the identity service's own responsibility (out of scope for this job, which only touches the lake).

The zero-rows edge case

An identifier with zero rows in a given table (e.g. present in raw but not yet propagated to curated) needs special handling: a `DELETE` matching nothing creates no new Iceberg snapshot, so the "pre-erasure" snapshot is simply the unchanged current one — which correctly still resolves. The job short-circuits on a zero-rows-before count and marks that table trivially successful rather than running the time-travel check at all (see bug #15, §10). This is a routine case in real usage, not a corner case worth ignoring — a real erasure request will rarely have rows in every table at once.

6 Erasure — Rehearsal Results

Two full rehearsals, the second one specifically to confirm the fixes held up under the **real deployed task role**, not just local credentials with broader access.

Rehearsal 1 — local, post-workgroup-fix

Table	Rows before	Rows after	Time travel
test_raw.trip_events	10	0	Blocked — snapshot 3011481733055550922 no longer resolves
test_curated.trip_events_curated	1	0	Blocked — snapshot 2915178321408864835 no longer resolves

Identifier hash_veh_000067 , jurisdiction IN , request ref rehearsal-2026-09-21-004 . **End-to-end duration: 28.03s** (brief step 1's "measure end-to-end erasure time"). Independently re-confirmed zero rows on both tables via a fresh Athena query afterward, not the job's own exit code.

Rehearsal 2 — real deployed task, real IAM role

Identifier hash_veh_001327 , request ref rehearsal-2026-09-21-008 , invoked via aws ecs run-task against the deployed mtsai-datalake-test-erasure task definition:

Table	Rows before	Rows after	Outcome
test_raw.trip_events	10	0	Erased, time travel blocked (19.37s)
test_curated.trip_events_curated	0	0	Correctly handled as "nothing to erase" (bug #15's fix)

This run exercised both code paths — a real erasure and the zero-rows short-circuit — in one deployed invocation. Independently reconfirmed 0 rows via a fresh Athena query after the fact.

Not tested: a real audit-triggered erasure request through an actual identity service process (none exists to trigger from) — every rehearsal here used a manually-supplied request reference and identifier, matching the brief's own "synthetic accounts" rehearsal scope for this phase.

7 Weekly Hygiene (`cmd/compact`)

Scheduled via `aws_scheduler_schedule.weekly_compact` , `cron(0 3 ? * SUN *)` UTC — Sunday 03:00, a low-traffic window off the nightly export/curate path. Runs `VACUUM` against both Iceberg tables, then publishes the cost dashboard's storage-by-prefix metric while it already has bucket read access.

Deployed verification

Run	Result
First deployed run (real task role)	Failed — <code>AccessDenied</code> listing the whole <code>athena-results/</code> zone (bug #16, \$10)
Second deployed run, after fix	Succeeded — both tables vacuumed, all 4 prefix metrics published, independently confirmed via <code>aws cloudwatch list-metrics</code>

Real measured sizes at last run: `raw/` 3.37 MB, `curated/` 186 KB, `athena-results/export/` 25 KB, `export-manifests/` 23 KB.

8 Cost Dashboard

New `terraform/modules/cost-dashboard` module, one `aws_cloudwatch_dashboard` with three widgets matching the brief's exact list (Section 7 Phase 3 step 4):

Widget	Source	Notes
S3 storage by prefix	Custom metric <code>MTSAiDataLake/Storage / PrefixBytes</code> , published weekly by <code>cmd/compact</code>	Native S3 CloudWatch metrics are bucket-level only, not per-prefix, without paid Storage Lens advanced metrics
Athena bytes scanned by workgroup	Native <code>AWS/Athena</code> <code>ProcessedBytes</code> , dimensioned by <code>WorkGroup</code>	No custom code needed — already existed once the pipeline had its own workgroup (\$4)
Fargate run duration by day	CloudWatch Logs Insights query on the shared export log group	Parses the <code>duration=X.XXs</code> every job (export/curate/compact/erasure) already logs on success — reuses data that already exists instead of a second tracking mechanism

URL: `https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#dashboards:name=mtsai-datalake-test-cost`

9 Budgets & Alerting — The Confirmation Saga

Three per-consumer `aws_budgets_budget` resources (\$50/\$50/\$25 monthly for analytics/forecasting/audit) and the export/curate failure-alert SNS subscription had existed in Terraform since earlier phases — both gated on `alarm_email` being non-empty, which it never was until this phase.

Getting the confirmation email to actually land took real troubleshooting

- 1 Set `alarm_email`, applied. AWS's SNS confirmation email **never arrived** at `siby@miracletraffic.ai`.
- 2 Tried two more addresses on the same domain (`romysiby@`, `romy@miracletraffic.ai`) — neither arrived either, ruling out a single-mailbox issue.
- 3 Tried a personal Gmail address specifically to test whether the `miracletraffic.ai` domain itself was filtering AWS's sending domain — also never arrived, ruling out domain-specific filtering.
- 4 Checked SNS's own diagnostics: delivery-status logging exists for SQS/Lambda/HTTP/Firehose/mobile push subscriptions, but **not for email** — there was no log to inspect, only AWS's own `PendingConfirmation` status with no further detail.
- 5 Pivoted to SMS as an alternative (no confirmation step at all) — also failed silently, this time for a genuinely diagnosable reason: the account's default **\$1/month SNS SMS spend limit**, confirmed via `aws sns get-sms-attributes`.
- 6 Root cause, once actually checked: the email had been arriving the whole time — landing in spam, every time, across all four addresses. Confirmed legitimate (sender `no-reply@sns.amazonaws.com`, topic ARN matched exactly), marked not-spam, confirmed.

Confirmed working, not just confirmed subscribed: published a real test message to the topic and received it at `siby@miracletraffic.ai`. Three other addresses tried along the way remain `PendingConfirmation` — harmless, AWS expires unconfirmed subscriptions on its own; not actively cleaned up.

10 Bugs Found & Fixed

Three real bugs this phase, bringing the project total to sixteen (see the companion Build Report for bugs 1-13). None caught by `terraform plan`, `go build`, or code review.

14. Every Athena query in the project was reading a stale cached Iceberg snapshot. No code specified a workgroup, defaulting to "primary" — reproducibly undercounted a live table by 11 rows. Full account in §4. Fixed with a dedicated Athena workgroup and an explicit `WorkGroup` field on every `AthenaRunner` call across all four binaries.

15. Erasing an identifier with zero rows in a table wrongly failed the time-travel check. A no-op `DELETE` creates no new Iceberg snapshot, so the "pre-erasure" snapshot is the unchanged current one, which correctly still resolves — the job was treating that as "not blocked" when there was nothing to hide in the first place. Fixed by short-circuiting on zero-rows-before as a trivial success.

16. `cmd/compact`'s first deployed run failed listing the whole `athena-results/` zone. Its `ListBucket` grant is deliberately scoped to only its own `athena-results/export/` subfolder, not other consumer workgroups' results — the same class of key-enumeration leak already fixed once in Phase 1 (bug #1). Fixed by narrowing the job's own ambition to match the already-correct permission, not by widening the grant.

All three fixes are applied and re-verified in Test — bugs #14 and #16 specifically re-confirmed under the real deployed IAM role, not just local credentials.

11 Verification Results

Check	Result
Erasure rehearsal 1 (local, post-fix)	10+1 rows erased across both tables, both time-travel-blocked, 28.03s, independently reconfirmed
Erasure rehearsal 2 (deployed, real IAM role)	10 rows erased in raw, curated correctly handled as zero-rows, independently reconfirmed
<code>cmd/compact</code> , deployed, real IAM role	Both tables vacuumed, 4 storage metrics published, confirmed via <code>list-metrics</code>
Budget alarms	3/3 created (<code>aws budgets describe-budgets</code>)
SNS failure alert	Subscription Confirmed (not <code>PendingConfirmation</code>); real test message published and received
Terraform drift	<code>terraform plan</code> clean after every apply this phase

12 Outstanding Items

Item	Why it's not done
Erasure for the other three personal-data tables	<code>accounts</code> , <code>anpr_camera_events</code> , <code>reward_ledger</code> aren't in the lake yet — nothing to build until Phase 2's second-table work happens.
Real audit-triggered erasure	No identity service integration exists to trigger from; every rehearsal used a manually-supplied request.
Root cause of the workgroup staleness	Reproducible and fixed, but the underlying AWS mechanism (why "primary" specifically cached stale Iceberg metadata) was never fully explained by available documentation.
SMS alerting	Blocked by the account's \$1/month spend cap; not raised since email ended up working.
KMS key policy restriction (brief Section 6)	Still relies on IAM alone, unchanged from Phase 1 - not a Phase 3 deliverable but still open.

Generated from the live state of the `mtsai-datalake` repository and the deployed Test AWS account (690293068614) as of 21 September 2026. Diagrams reflect verified behavior, not aspirational design. Companion to `MTSAi-Data-Lake-Build-Report.pdf` (Phases 0-2) and `MTSAi-Data-Lake-Flow.pdf` (plain-language flow).